

Compte-rendu 5

Introduction

Dans le cadre du projet de programmation de l'UE HAI809I, nous devons générer des images mosaïques en appliquant des critères avancés. Ce rapport présente les avancées réalisées dans ce domaine durant la dernière semaine de mars.

Table des matières

1	Base de code et sécurité	2
1.1	Image	2
1.2	ImagePGM et ImagePPM	3
1.3	Figures	4
2	Génération d'image	5
2.1	Optimisations	5
2.2	Interaction utilisateur	6

1 Base de code et sécurité

Introduction

La base de code en C++ a été complétée. En effet, celle-ci se compose de 3 Class Image, ImagePGM et ImagePPM facilitant le traitement des Images.

1.1 Image

Cette classe, sert d'interface au deux autres. Elle implémente les opérations de base sur la largeur et la hauteur des images.

```
1      class Image {
2          protected:
3              size_t width;
4              size_t height;
5
6          public:
7
8              /* Constructeurs et destructeurs */
9              Image();
10             Image(size_t newWidth, size_t newHeight
11                 );
12             virtual ~Image(); // Destructeur
13                 virtuel
14
15             /* Accesseurs et opérateurs d'accès */
16             size_t getWidth();
17             size_t getHeight();
18             size_t size();
19             virtual octet* operator[] (size_t i);
20                 // Opérateur virtuel
21
22             /* Setter virtuel */
23             virtual void resize(size_t newWidth,
24                 size_t newHeight);
25             virtual void segment(size_t blockSize);
26
27             /* Chiffrement */
28             virtual size_t* swap(size_t blockSize);
29             virtual void sort(size_t* key, size_t
30                 blockSize);
31
32             /* Lecture et écriture dans des
33                 fichiers */
34             virtual void read(const char* path);
35             virtual void write(const char* path,
36                 const char* comment = nullptr);
37     };
```

```
31  
32      uint32_t getNanoSecondSeed();
```

Listing 1 – Classe Image

1.2 ImagePGM et ImagePPM

Outre certaines spécificités, les implémentations des classes PGM et PPM est sont sensiblement les même. Elles se composent de certain nombre de méthodes et opérateurs permettant des opérations sur les deux format d'image.

- `octet* operator[] (size_t i)`: Opérateur permettant l'accès aux valeurs de l'image, simulant le comportement d'une matrice.
- `void operator= (const ImagePGM &other)`: Opérateur permettant de copier les valeurs et la taille d'une image dans une autre.
- `void resize (size_t newWidth, size_t newHeight)`: Méthode permettant de redimensionner des images en utilisant l'interpolation bilinéaire, nous évitant d'utiliser des bibliothèques d'images telles que OpenCV pour ce genre d'opérations.
- `void segment(size_t blockSize)`: Méthode permettant de segmenter l'image et d'en faire une image de blocs moyens de taille `blockSize`.
- `size_t* swap(size_t blockSize)`: Méthode permettant la permutation des blocs pour la mise en place d'un chiffrement par permutation, retournant la liste de permutation.
- `void sort(size_t* key, size_t blockSize)`: Méthode permettant de reformer l'image originale à partir de la liste de permutation.
- `void read(const char* path)`: Méthode permettant de lire une image à partir d'un fichier.
- `void write(const char* path, const char* comment = nullptr)`: Méthode permettant d'écrire une image avec un commentaire ou non (utile à l'avenir pour l'enregistrement du critère moyen).

1.3 Figures



Figure 1 – Image original



Figure 2 – Image redimensionnée.



Figure 3 – Image segmentée.

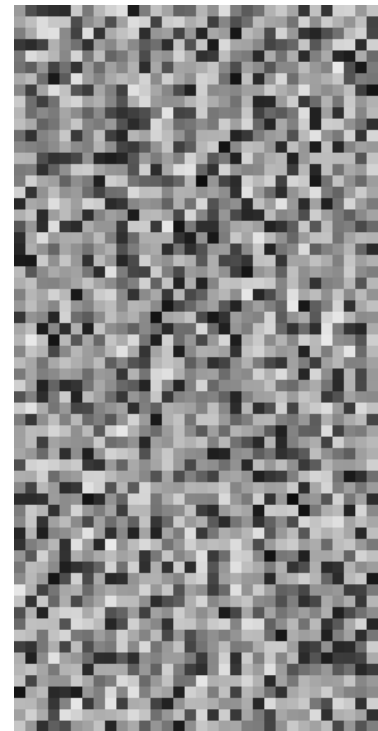


Figure 4 – Image permutée.

2 Génération d'image

2.1 Optimisations

Base d'image

La base d'images nouvellement créée étant de taille conséquente, le traitement de celle-ci peut être allongé à cause de divers facteurs (taille de la RAM, type de périphérique de stockage), il nous faut donc optimiser la manière dont nous utilisons et pré-traitons nos données.

Le premier choix a été de limiter la taille des imageriettes à des formats précis (4x4, 8x8, 16x16, 32x32, 64x64, 128x128) afin de limiter la taille en mémoire, mais aussi d'éviter la nécessité de recalculer les critères des imageriettes.

De plus, la création de la base d'imageriettes et le calcul de critères se font avec du multi-threading afin d'accélérer le processus.

Ces solutions nous ont permis de passer d'une base d'imageriettes de 100 Go à une base d'imageriettes de 10 Go, ce qui, à l'aide de la parallélisation nous permet de réduire significativement le temps de pré-traitement des imageriettes (sur un ordinateur équipé d'un SSD, l'opération prend moins de 10 minutes). Avoir généré toutes les tailles d'imageriettes nous permet de calculer les critères une unique fois, puis de les stocker dans un document CSV (plus léger qu'un json, et plus lisible qu'un txt), ce qui évite des calculs pour les futures utilisations.

Génération d'une image mosaïque

Nous avons également pu apporter des améliorations à la génération d'une image mosaïque. Tout d'abord, nous ne convertissons plus les images PPM en PGM, mais utilisons simplement la formule de la luminance :

$$Y = 0.299R + 0.587G + 0.114B$$

Pour la génération de l'image, nous laissions précédemment le choix à l'utilisateur de la taille des blocs en modifiant le code. A présent, ce dernier se verra proposer des tailles de blocs en fonction de la qualité souhaitée (prédéfinie manuellement en fonction du nombre de blocs par ligne.). L'utilisateur peut alors choisir l'image et la taille de bloc de son choix, le programme redimensionnera automatiquement l'image afin qu'elle corresponde à la taille d'un bloc :

$$nH' = \left\lfloor \frac{nH}{TAILLE_IMAGETTE} \right\rfloor \times TAILLE_IMAGETTE$$

$$nW' = \left\lfloor \frac{nW}{TAILLE_IMAGETTE} \right\rfloor \times TAILLE_IMAGETTE$$

2.2 Interaction utilisateur

L'utilisateur n'a désormais plus besoin de modifier le code afin d'utiliser le programme, il peut saisir le chemin de l'image PPM de son choix et choisir la taille de l'image directement depuis le terminal. Il a également la possibilité de faire plusieurs images sans avoir à relancer le programme.